GNN Lab

Environment Setup

This document outlines the structure of the laboratory assignment.
The project depends on several external libraries listed in the requirements.txt file.

Setup Instructions:
To ensure compatibility and proper execution, please create a virtual environment and install the dependencies:

1. Open your terminal in the project folder.
2. Create a virtual environment: python -m venv .env
3. Activate the environment:
    a. Windows: .env\Scripts\activate
    b. Linux/Mac: source .env/bin/activate
4. Install dependencies: pip install -r requirements.txt

Once installed, you can open the notebook in VSCode by typing **code main.ipynb**.

Note: This assignment was developed using Python 3.12. Using significantly older versions or different 3.x releases may cause compatibility issues.

Files Structure

The following section provides a high-level overview of the files attached.

cosine_center_loss.py

Implements Cosine Center Loss, a geometric regularization technique designed to minimize intra-class variance by pulling node embeddings toward their corresponding learnable class centers. Instead of relying on contrastive comparisons between samples (which requires massive batch sizes to find enough negative examples), this approach acts directly on the structure of the latent space. It enforces spherical normalization on both the embeddings and the class centers, ensuring that the optimization is driven only by the angular direction rather than vector magnitude, making the model more robust to outliers and stable on small datasets. The loss is minimized when the embedding aligns perfectly with its class center (cosine similarity = 1).

The actual loss is:

$$L = \frac{1}{N}\sum_{i=1}^{N}\left(1 - \cos\left(\theta_{i,y_i}\right)\right) = \frac{1}{N}\sum_{i=1}^{N}\left(1 - x_i^T * c_{y_i}\right)$$

Where:

- N: batch size
- x_i: normalized vector of index i
- c_{y_i}: normalized center vector for the class i

This approach belongs to Deep Metric Learning; a family of methods that teaches the model to map similar data points close together in geometric space. When paired with Cross-Entropy, it achieves a 'push-pull' effect like contrastive learning: CE pushes different classes apart, CCL pulls samples of the same class toward their center. This is also the reason why, for simplicity, in this report, I talk about a contrastive approach.

The module instantiates two class centers randomly on a hypersfere then works by updating their position so that each vector has the same direction of its center.

Please note that this loss by itself can “cheat” and map every embedding and center to the same point. This is why it is never used in isolation but rather added as a regularization term.

models.py

This module implements the deep learning models designed for the node classification task. It provides two main classes:

1. **MLP (Baseline):** a standard Multi-Layer Perceptron that processes each node independently. It serves as a baseline to verify how much performance is gained by using graph topology, as this model ignores the connections between nodes and looks only at their features.

2. **GNN:** it is a flexible architecture that can swap between different graph layers to process both node features and the graph structure.

The unique feature of the GNN class is its dual-objective training. When the contrastive mode is enabled, the model doesn't just try to classify nodes correctly; it also forces the internal embeddings to be geometrically structured. The parameter alpha controls the balance between these two goals. To make the Cosine Center Loss work effectively, the code includes a specialized method (get_optimizer_params): this splits the model parameters into two groups during training: the network trains with a standard learning rate and weight decay, while the class centers are given a higher learning rate so they can quickly move to the correct positions in the vector space, and weight decay is disabled to prevent them from shrinking towards zero.

The implementation allows for modularity, allowing different architectural swaps by changing only a few parameters:

- **Classification Head (Linear vs. MLP):**
  - **Linear:** Projects the GNN features directly to the class logits.
    - *Pros:* low parameter count and high interpretability. It forces the GNN encoder to produce linearly separable features.
    - *Cons:* cannot solve complex non-linear boundaries if the GNN fails to use the graph structure efficiently.
  - **MLP:** Adds a non-linear hidden layer before classification.
    - *Pros:* higher capacity to model complex decision boundaries.
    - *Cons:* risk of overfitting on small datasets. It may also decouple the latent space from the classification task, making the geometric regularization of the Center Loss less effective on the final decision.
- **Encoder Variants:**
  - **GCN:** effective as a strong baseline for global graph structures.
  - **GAT:** uses attention mechanisms to weigh edges differently. Excellent for noisy graphs. High parameter count, unstable to train on very small datasets.

utils.py

This file contains the helper functions I used to train the models.

1. **train_model_one_fold.** This function serves as the standard training pipeline for a single model configuration. I designed it to handle the specific requirements of the contrastive learning architecture.
   a. It calls get_optimizer_params instead of the standard model.parameters(). This is necessary to apply different learning rates and weight decay settings to the neural network weights versus the loss centroids.
   b. This function uses the latent representation h to calculate the **Silhouette Score**. This allows me to quantitatively measure how well the model clusters the classes in the embedding space, not just its classification success.

c. It implements early stopping with model checkpointing (best_state), ensuring that the final evaluation is performed on the best version of the model rather than the last epoch, avoiding overfitting.

2. **plot_analysis**. This function prints a boxplot of the specified metric for all the models inside the result dictionary. It allows me to visually inspect my results, noticing any clear winners immediately.
3. **get_stats.** This function calculates mean, standard error and CI95% by applying the Nadeau and Bengio Correction. It is necessary because the collected samples are obtained by multiple folds of the same data.

$$CI = \mu \pm t \cdot \sqrt{\left(\frac{1}{N} + \frac{n_{test}}{n_{train}}\right)\sigma^2}$$

4. **ttest_corrected**. This function is mainly used for the computation of the p-value by applying a parametric test and calculating the t-statistics. This also implements the Nadeau and Bengio Correction. The p-value is then calculated using the provided scipy function.

$$t = \frac{\overline{(d)}}{\sqrt{\left(\frac{1}{N} + \frac{n_{test}}{n_{train}}\right)\sigma_d^2}}$$

Where $d = metric(model_1) - metric(model_2)$

main.ipynb

This notebook is divided into several sections; each section is described in the following chapter while I comment all my operations and findings.

- Section 0: contains all imports
- Section 1: dataset loading
- Section 2: EDA.
- Section 3: graph construction
- Section 4: model(s) instantiation and train – test loop.

Assignment

My approach for this assignment is to compare different architectures, to see if a MLP can work better than a GNN (or vice-versa) and to understand if a more clustered embedding space can improve the model accuracy.

I fixed a hyperparameter baseline (i.e. the number of layers of the network, the dropout) and then I tried to understand if changing those influences the result. My goal was not to find the best hyperparameters to "squeeze" the most out of my data, but to understand if a regularization term was useful.

All the choices I made were influenced by the results obtained in the previous steps.

Dataset Exploration

The very first thing to do is to check the size of the dataset and some statistics.

Upon inspection, I saw that the class label is a string (here shown as object) and the rest of the file are numbers. These values appear as decimals even in the raw file because the dataset does not contain manual counts, but **statistical estimates** generated by bioinformatics software, resulting in decimal values.

Dataset Shape: 100 Samples (Rows) x 1023 Columns
Data Types Summary:
float64 938
int64 84
object 1

I then checked if some preprocessing was needed:

- I saw there are no duplicates but also that almost 50% of the matrix is 0, this implies data sparsity and it is common when working with gene expression data.
- It is also very common to have a gigantic range (0 – 3billion), which shows me that I need a logarithmic compression of the data. There are 84 columns (genes) that give no relevant information. Both issues are addressed later.

Integrity Check		
Missing values	Duplicate Rows	
0	0	
Global Statistics		
Total Cells	Total Zeros	Global Sparsity
102200	47513	46.49%
Min Value	Max Value	Silent Genes
0	3,214,496,710.67	84

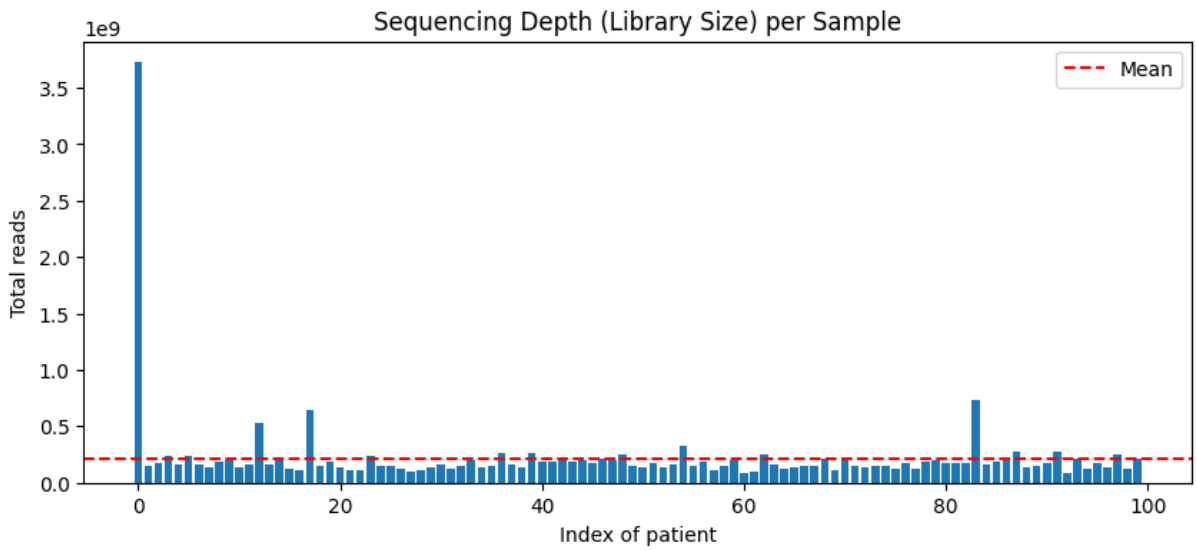
Next step was to see the number of counts for each patient (the library size).

Min	Max	Average
83,080,847.21	3,723,676,510.92	217,901,779.66

As shown in the plot below, the patient at index 0 exhibits an extremely high library size (~3.7 billion vs an average of 218 million). This is usually caused by technical choices made in the laboratory while sequencing data, not over-expression.

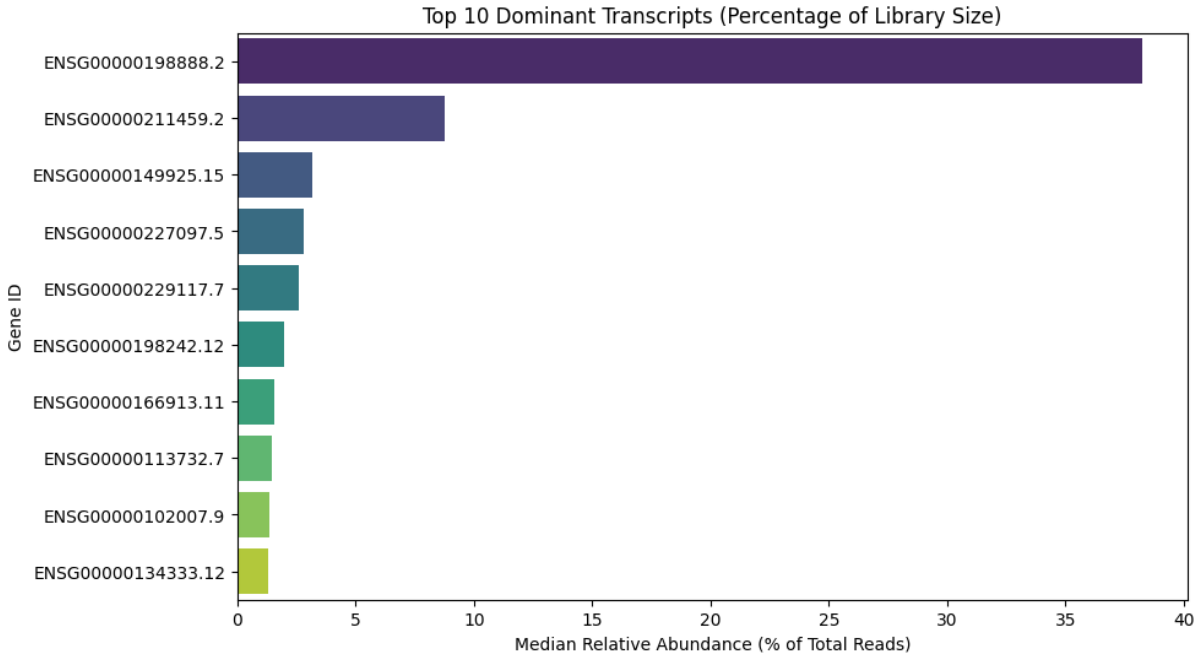
I considered normalizing the data using the Counts-Per-Million (CPM) method (dividing each row by the sum of the row itself times one million), but ultimately decided against it.

While CPM adjusts for total read counts, it forces the data into proportions that must sum to a fixed value. Since my dataset consists of a random subset of only 1022 genes, the total count used for this calculation is unstable and can vary unpredictably. Using CPM risks creating artificial patterns (i.e. making some genes appear less expressed simply because others are high) which would distort the signals needed for classification.

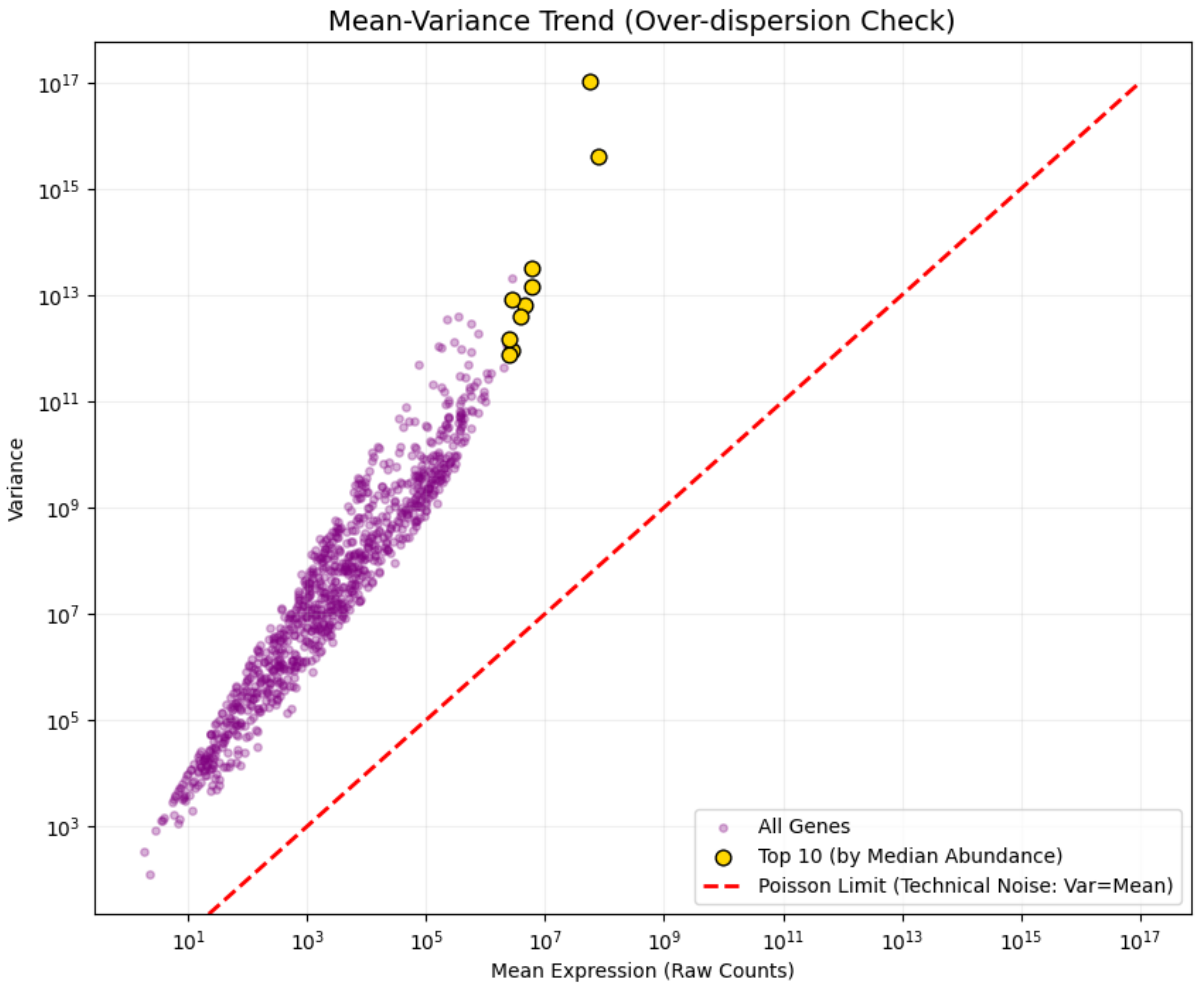


The next step was to identify the most highly expressed genes to detect any dominant transcripts (the few genes that monopolize the total reads). I computed each gene's relative abundance (as a % of total reads for a patient) across all samples and selected the top 10 dominant transcripts. I used the median because it's robust and reduces the influence of the single extreme outlier sample.

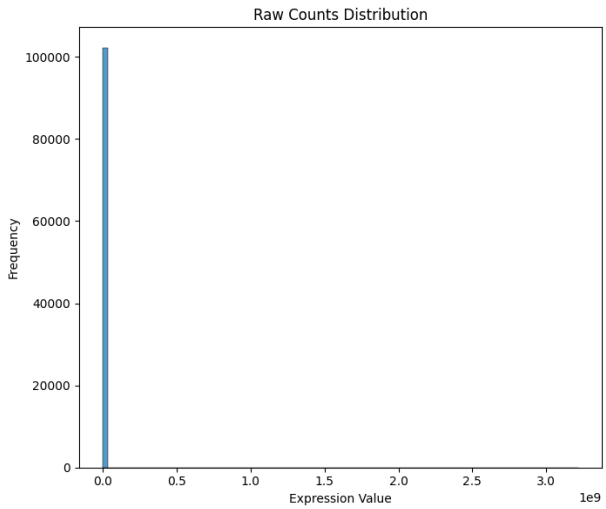
By applying CPM, variations in this single high-abundance gene would mathematically lower all other 999 genes, so this is another reason not to apply it.

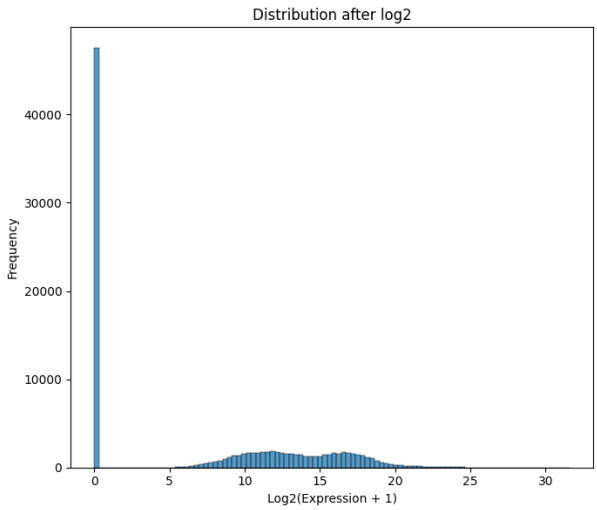


The next step was to assess the variability of the dataset to ensure it contained retrievable signals rather than just noise. I plotted the variance against the mean for all genes, comparing them to the Poisson limit (red line), which represents random noise. All genes falling above this line indicates **'over-dispersion,'** confirming the presence of biological variation.

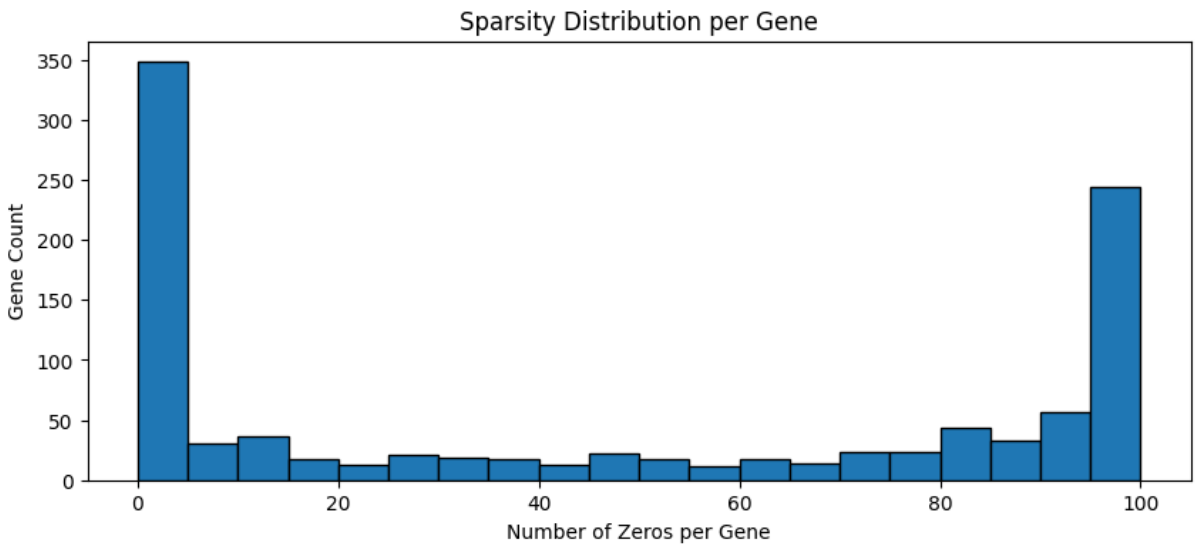


Then, I plotted the distribution of all expression values to see how the data behaves. In the first plot, the data is heavily skewed: most genes have very low values, while a few 'giant' genes dwarf everything else. This makes everything disappear on the X-axis except for the high amount of zeros. To fix it, I applied a Log2 transformation. As shown in the second plot, the values are now much more spread out.





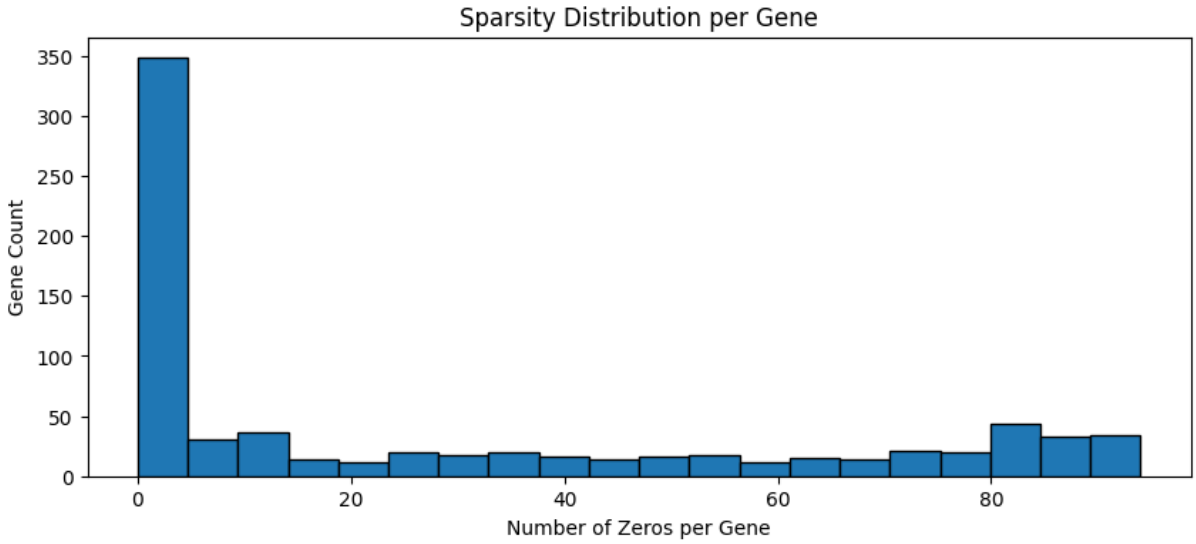
Given that this is a very small dataset, it is fundamental to me to reduce the amount of noise because otherwise the networks will overfit immediately. So for each column (gene), I counted how many times it was zero across all patients. Then, I made a plot to show how many genes share the same number of zeros. For example, the first bar shows that about 340 genes have from 0 to 4 'zeros', meaning they are active in (almost) every patient. On the other hand, the bars on the far-right show genes that are almost always zero and are therefore not useful.



Given this situation I decided to remove not only all the genes that are not expressed in any patient, but I also decided to remove the “almost dead” genes (genes that are at 0 in >=95% of patients). I did this by applying a class-unaware criterion: a gene is kept if it is expressed (> 0)

in at least 5% of the patients. In this way I can remove the columns on the right side of the histogram (extremely sparse genes).

After this filtering step, I am left with 793 genes.



I checked for class imbalance to see if I needed specific techniques to handle a skewed distribution. I can confirm that the classes are perfectly balanced (50 samples for Luminal A and 50 for Luminal B), which is ideal for training the model.

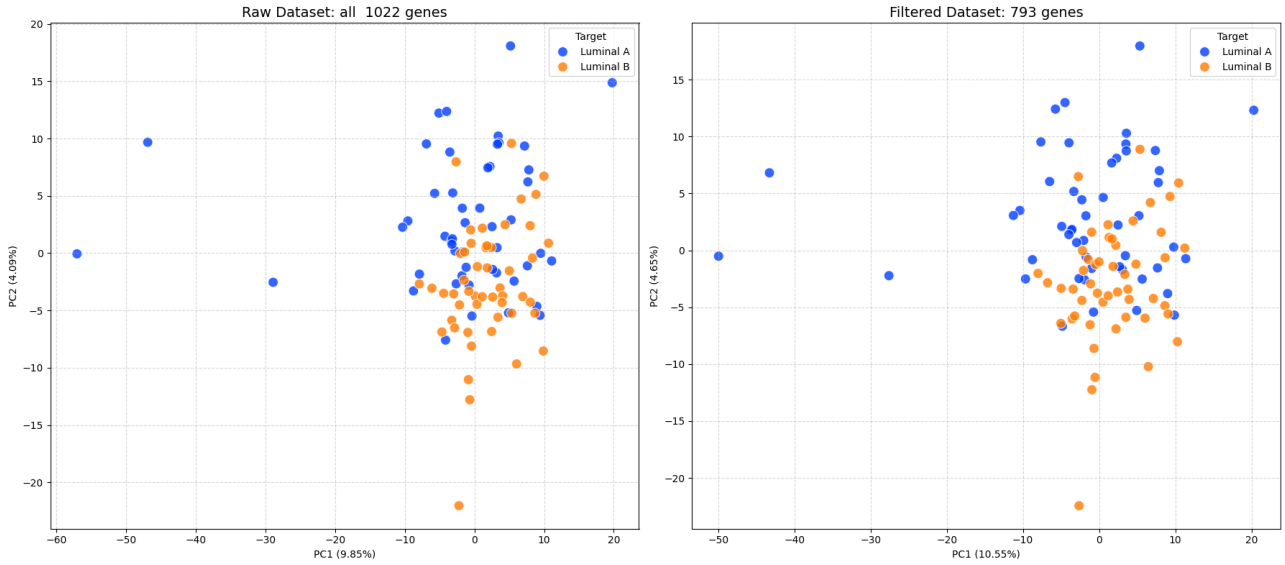
Class distribution	
Luminal A	50
Luminal B	50

I created these PCA plots to compare the dataset before and after the filtering step (raw dataset: 1022 genes vs filtered dataset: 793 genes).

In both cases, Luminal A and Luminal B show a similar overall pattern: there is a partial separation mainly along PC2, with Luminal A samples tending to have higher PC2 values although there is still overlap in the central region and a few outliers.

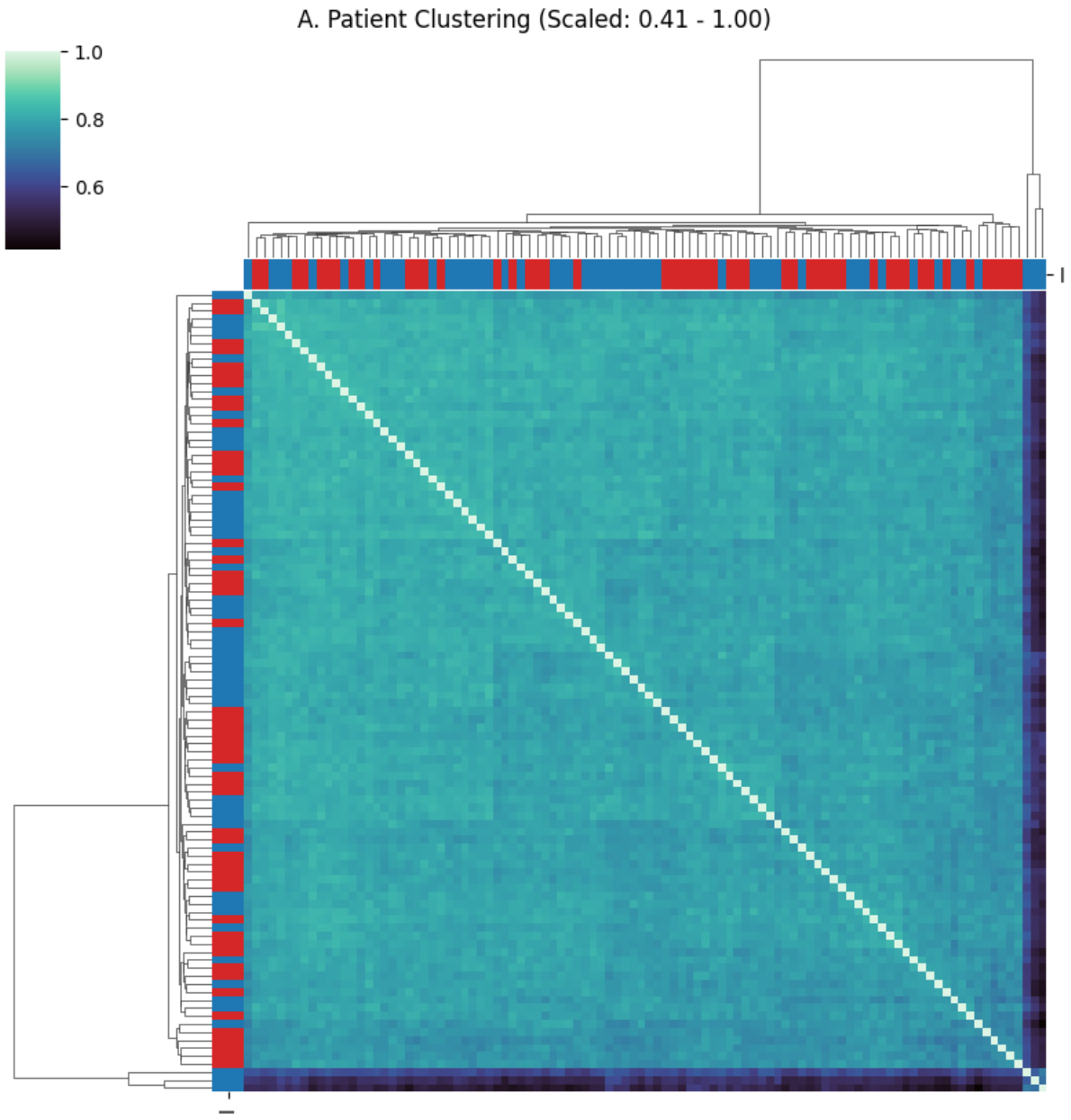
After filtering, the PCA structure remains consistent with the raw data, suggesting that removing very sparse genes does not change the patterns, but mainly reduces the feature space.

It is important to note that PC1 and PC2 before and after filtering are not exactly the same axes. PCA components are defined as the eigenvectors of the covariance matrix of the features, so changing the set of genes changes the covariance structure and can slightly rotate the principal directions (the axes). Therefore, the explained variance ratios of PC1 and PC2 can also change slightly, even when the visual separation looks similar (PC1 and PC2 represents the proportion of variance captured by each individual principal component relative to the total variance of the original dataset).



After the PCA, I used a **Clustermap** to better understand how similar the patients are to each other. This plot automatically groups patients with similar genetic profiles, hoping that patients of the same class are very similar.

I added color bars on the sides (Blue for Luminal A, Red for Luminal B) to check if patients of the same type clustered together. Like PCA results, I observed that the two subtypes are quite mixed, with no clear 'blocks' of a single color.


A. Patient Clustering (Scaled: 0.41 - 1.00)
1.0
0.8
0.6

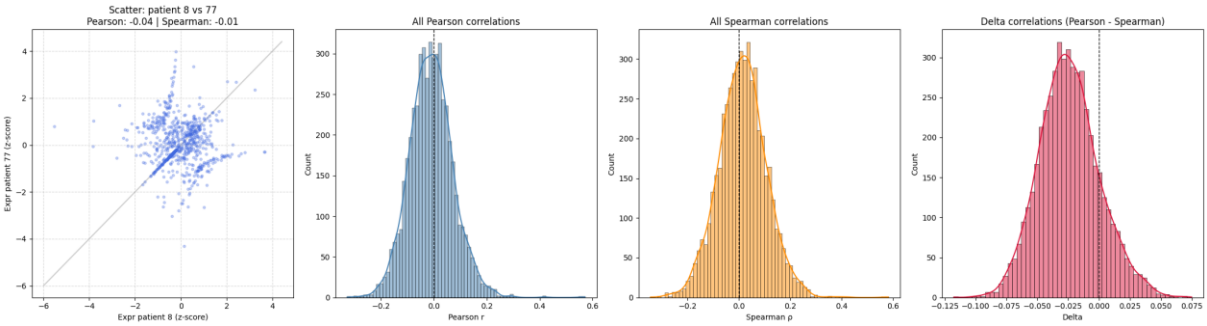
Graph Construction

The next step is graph building. This is essential because a GNN requires information to flow through neighbors during message passing.

Before computing patient–patient similarities, I standardized the expression matrix gene-wise. This brings each gene to zero mean and unit variance, making genes directly comparable and reducing the risk that a small subset of high-variance genes dominates both the similarity computation and the neural models; moreover, scaled inputs are generally beneficial for neural-network optimization stability.

I did this to prevents genes with high variance from dominating the similarity metric, ensuring all variations in genes contribute equally. A possible drawback is that z-scoring can amplify noise for genes with very small variance (but this is mitigated by the gene filtering) and also changes the clustermap shown before.

After gene-wise z-score standardization, Pearson and Spearman patient–patient correlations show very similar distributions. Since the two measures are comparable in this setting, I used Pearson correlation as the default similarity metric for graph construction for simplicity, while noting that Spearman is a valid alternative.



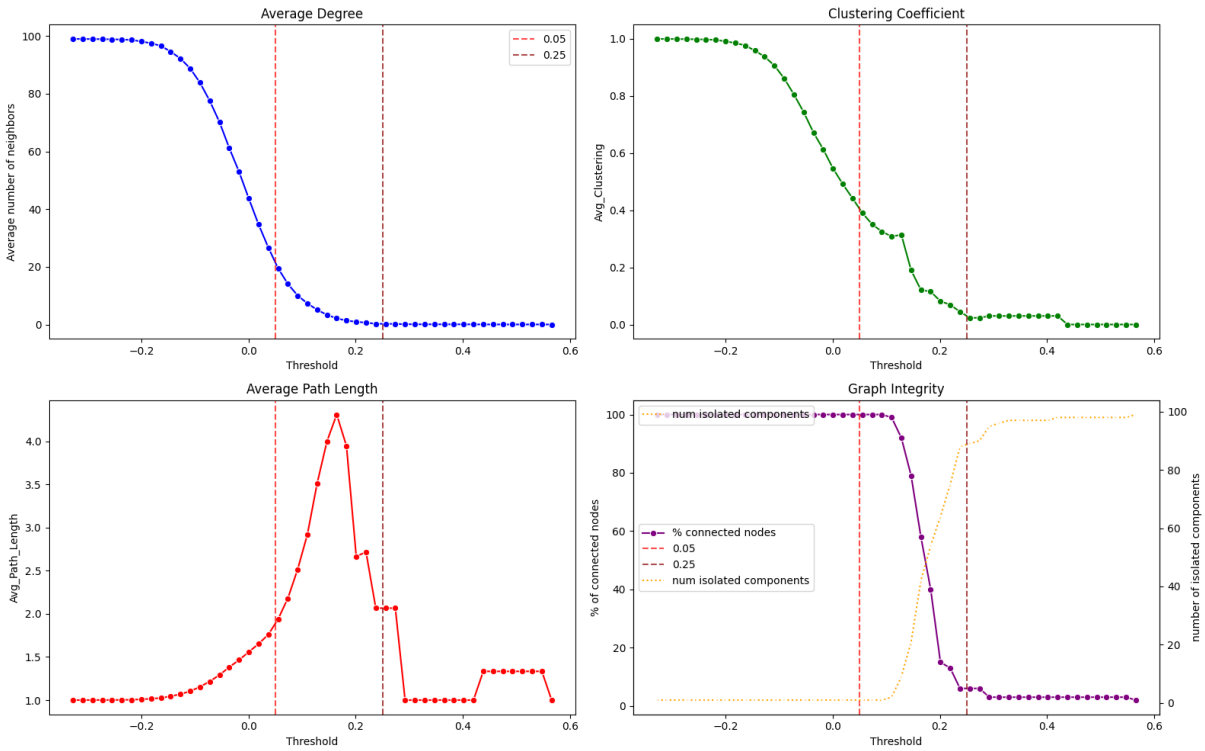
Instead of building edges using a fixed correlation threshold, I used a kNN graph. A hard threshold can easily make the graph fragment into multiple connected components once the threshold becomes too strict; this issue would appear with both Pearson and Spearman. With kNN, each node keeps a fixed number of neighbors, which makes the graph structure more stable for message passing and avoids extreme fragmentation.

To justify this choice, I first explored what happens when edges are built by thresholding the patient–patient correlation at a value t. This analysis was done on Pearson.

Therefore, I calculated several graph metrics as a function of the threshold t:

- **num_edges:** Counts the total number of edges in the graph.
- **avg_degree:** Measures the average number of neighbors each node has.

- **avg_clustering:** Measures the compactness of the graph. Specifically, it represents the fraction of pairs of a node's neighbors that are also connected to each other (forming a triangle). A higher value indicates dense local connectivity.
- **avg_path_len:** Measures the average number of "jumps" (edges) required to travel from one node to another. This is calculated on the Largest Connected Component (LCC) to avoid infinite distances in disconnected graphs.
- **n_components:** Counts the number of disconnected subgraphs (isolated groups of patients).
- **nodes_in_lcc_pct:** Measures the percentage of nodes belonging to the Largest Connected Component (LCC). This indicates how much of the dataset remains connected in the main "giant component".

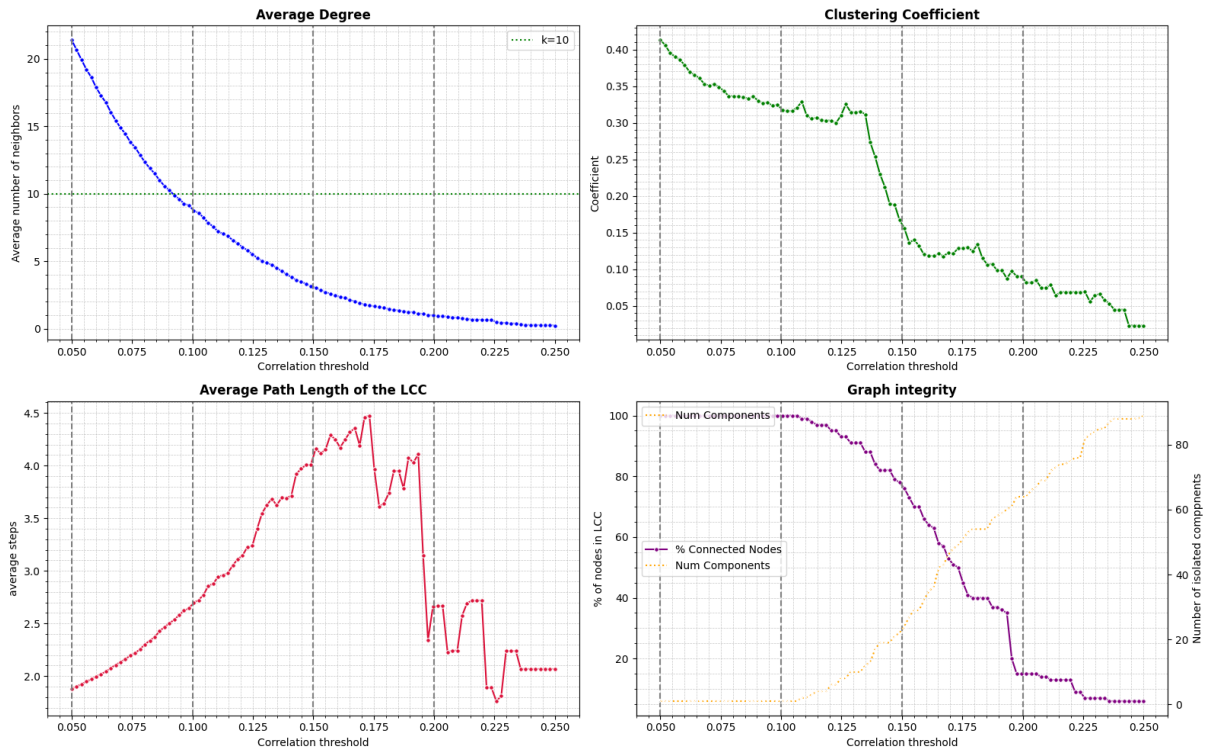


Let's contextualize these metrics. To help the analysis, I plotted two vertical lines at the thresholds 0.05 and 0.25.

- **avg_degree:** Starts dropping significantly around t = -0.10. At t = 0.00, the degree is still quite high (~40), which might cause oversmoothing. The target range of 5–15 neighbors (the "sweet spot" for message passing) is reached between 0.05 and 0.20. Beyond 0.20, the degree collapses to near zero.
- **avg_clustering:** Remains close to 1.0 until t = -0.10, indicating that the graph is initially composed of dense cliques (everything is connected to everything). The metric starts to drop after, indicating the breaking of these tight triangles. We want to select a threshold where the clustering is not 1.0 (to reduce redundancy) but has not yet collapsed to 0.

- **avg_path_length:** Remains flat at 1.0 for a long time, confirming that initially, most nodes are directly connected. The spike at **t = 0.15** represents the phase where the graph becomes sparse, forcing information to travel through multiple hops to reach other nodes. This is desirable for a GNN to learn complex features, provided the graph remains connected. The subsequent drop to 0 means the graph is breaking apart.
- **Graph Integrity (LCC & Components):** The graph maintains >90% of nodes in the Largest Connected Component (LCC) up to **t = 0.10**. After this point, the network "shatters": the percentage of connected nodes drops vertically, and the number of isolated components skyrockets.

Based on these observations, I needed to repeat the analysis in this smaller range to see better what happens.



Although there is a range of thresholds that gives a reasonable average degree (t=0.10), a fixed threshold is still rigid because it does not directly control how many neighbors **each** node gets patients with globally lower correlations can end up with very few neighbors. To better control sparsity and ensure every node has a minimum neighborhood for message passing, I therefore switched to a k-Nearest Neighbors (kNN) graph: by construction, each node connects to its k most similar neighbors, making the graph density more stable and reducing the risk of isolated nodes.

The next step is to compare different graph setups. My goal is to maximize the information flow while keeping the graph clean and efficient. To find the best structure, I tested different numbers of neighbors (k=5, 10, 15) using both **Directed** and **Undirected** modes.

- **Directed** graphs are risky for this type of data. A patient might choose 5 neighbors, but if nobody chooses them back, they receive no information from the network. The **Undirected** mode solves this problem, enforcing a two-way connection. In practice, it starts by creating directed edges and then turns them into two-way connections (symmetric)
- **k=5 vs. k=10:** the results showed that k=10 achieves 100% connectivity, exactly like k=5. However, k=10 doubles the number of edges. In a GNN, adding unnecessary edges can introduce noise.

To conclude, I selected the **Undirected k=5** configuration. Since both k=5 and k=10 successfully connect all patients, the simpler option (k=5) is the best choice. It guarantees full connectivity with the minimum amount of noise.

GRAPH RECAP	
Nodes:	100
Features:	793
Edges:	350
Average Degree:	7

The last metric I calculated is the homophily ratio.

- **Edge Homophily** (0.643) measures the percentage of connections between nodes that share the same class
- **Baseline Random** (0.500) indicates the probability of connecting nodes of the same class by chance
- **Adjusted Signal** (0.143) represents the true informative value of the graph, obtained by subtracting the random baseline from the edge score.
- **Local homophily:** there are 21 nodes with homophily < 0.5, meaning they are connected to at least 2 out of 5 elements of the other class. This makes them difficult to predict for a GNN.

The Adjusted Signal reveals that the graph structure provides a weak signal. Consequently, the standard aggregation process of GCN might still dilute valid features with noise from dissimilar neighbors.

I hypothesize that GAT could now use this signal, learning to filter out the noisy connections. However, given the very limited sample size there is a high risk that the attention mechanism may struggle to learn stable weights, potentially leading to overfitting.

Models Instantiation and Training

In this section I provide multiple setups and analysis of the results.

To ensure reproducibility and consistency given the data scarcity (100 total records), I employed a RepeatedStratifiedKFold strategy. This ensures that every combination of train/validation/test is evaluated multiple times, reducing the variance of the results. The folds are pre-generated, so each architecture is always tested on the same set of data, allowing me to train different models at different times but excluding one factor of randomness from the training.

I implemented an early stopping mechanism (evalutating the loss onto a small sample of the train fold, saving the weights of the model every time the loss decreases, and restoring the best state once it sees that the loss as not decreased for 20 epochs) to reduce the risk of overfitting. I am aware that, since I am applying transductive learning using binary masks, there will be some data leakage.

Regarding the evaluation metrics, I collected:

- **Accuracy:** The percentage of correct predictions, serving as the primary metric for classification performance.
- **Silhouette Score:** A metric ranging from -1 to 1 that evaluates the quality of the latent space. It measures how similar an object is to its own cluster compared to other clusters.

The silhouette score represents how clustered are the classes. Given they are projected onto a unitary sphere, I used the version of this metric that uses the cosine distance.

$$Distance(x, y) = 1 - cosine(x, y) = 1 - x * y$$

For two clusters, this metric calculates the average distance between a point i and all other points of its cluster (and calls it $a(i)$). Then it calculates the average distance between the same point i and all the points of the other cluster (and calls it $b(i)$).

The silhouette score for i is:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

The silhouette score is the average of all silhouette scores. It is particularly important to verify the effect of the Cosine Center Loss: a higher Silhouette Score would confirm that the model is learning to group samples of the same class more tightly, even if accuracy remains similar.

First, I analyzed the boxplots, then used these metrics to compute the confidence intervals and the p-values across all the models.

- The number of samples collected (100) allows me to use the Central Limit Theorem.
- The samples collected are not skewed towards the upper limit.

- These samples are not i.i.d. because they are collected from different folds of the same data, with train and test data overlapping multiple times.

These conditions allowed me to use parametric tests but also forced me to use the Nadeau and Bengio correction, implementing the functions from scratch, as described in the utils section.

Initial Setup - GCN

In this section, I experimented with multiple configurations to assess the models' behaviors under different constraints.

I established a fixed baseline of parameters as follows:

- **Type_layers**: GCN. Most of the time less is more and a simpler aggregation mechanism is the best starting point
- **Num_layers**: 2. This lets a node update its embedding based on the neighbors of its neighbors, using the information from around half of the graph. This avoids oversmoothing because it prevents the recursive aggregation from diluting node features into a global average.
- **Dropout, num_classes, input_dim, hidden_dim** are all set to standard values for both the MLP and the GNN(s).

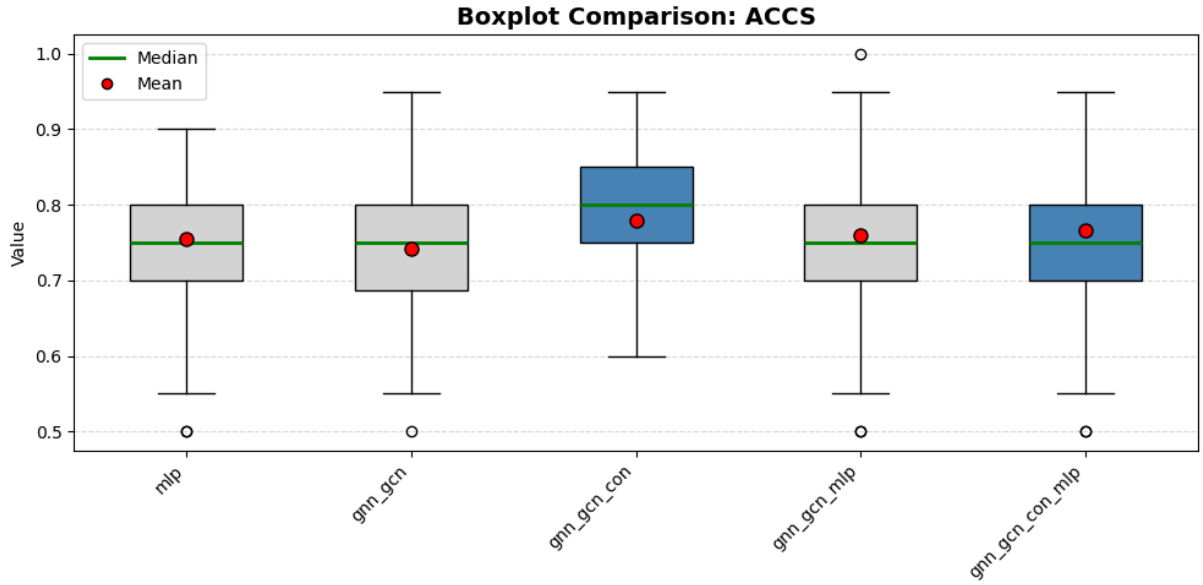
I tested the different behaviours of 2 different hyperparameters and compared these four combinations against a standard MLP baseline

- **Contrastive:** False / True. This flag enables the computation of the cosine center loss. I set its weight alpha to 0.7 relative to the Cross-Entropy loss, prioritizing the classification task while using the contrastive signal as a regularizer. This flag also enables the learning rate of the centers that need to adapt their position during training.
- **Classifier:** Linear / MLP. This represents the final stage of the network. A **Linear** layer creates a simple decision boundary in the embedding space, whereas an **MLP** can model non-linear relationships, though at a higher risk of overfitting. If the MLP is used, its hidden layer is given the same number of neurons as the MLP baseline.

Starting from the hypothesis that the graph provides a non-negligible signal my goal was to determine if the described hyperparameters could regularize the embedding space, improving classification performance.

I organized these combinations into a configuration dictionary, keeping all other parameters at their default values to isolate the impact of these specific variables.

Accuracy



This boxplot provides a visual comparison of the models' accuracy distributions.

- **Median (green line):** the gnn_gcn_con model displays a slightly higher median, indicating a superior typical performance. The gnn_gcn_con_mlp variant, however, aligns its median with the standard MLP and GCN baselines.
- **Mean (red dot):** the MLP is balanced; the gnn_gcn_con has a mean that is slightly lower than the median: this indicates that most of the time it performs well but there are a few outliers lowering its performance
- **IQR (the box):** the gnn_gcn_con box is positioned higher overall, implying that the middle 50% of its runs are consistently better than the baselines: we can clearly notice its 25th quartile is at the same level as the typical performance of other models. This improvement is not visible for the gnn_gcn_con_mlp, suggesting that the increase in parameters doesn't help.
- **Outliers (empty dots):** The gnn_gcn_con shows more stability with no outliers under 0.6, while all other models have wider whiskers or one or more outlier.

This plot shows that adding contrastive modules helps performance-wise, but only for the simpler model: the mlp after the encoder brings no real value. The actual numbers confirm the analysis.

MODEL	MEDIAN	MEAN	STD	CI 95%
mlp	0.7500	0.7555	0.0867	0.0877
gnn_gcn	0.7500	0.7420	0.0968	0.0980
gnn_gcn_con	0.8000	0.7790	0.0856	0.0866
gnn_gcn_mlp	0.7500	0.7600	0.0969	0.0981
gnn_gcn_con_mlp	0.7500	0.7660	0.0899	0.0909

The gnn_gcn_con model has the highest mean accuracy; it beats both the standard MLP and the basic GCN

The Standard Deviation is similar across all models. The contrastive models have a variance slightly lower than the simple MLP. This means the new method improves accuracy without making the model less stable.

The Confidence Intervals are wide and overlap completely. For example, the MLP interval covers the same range as the best models. This suggests that the performance difference between models is small.

I also performed the paired t-test between all couples of models.

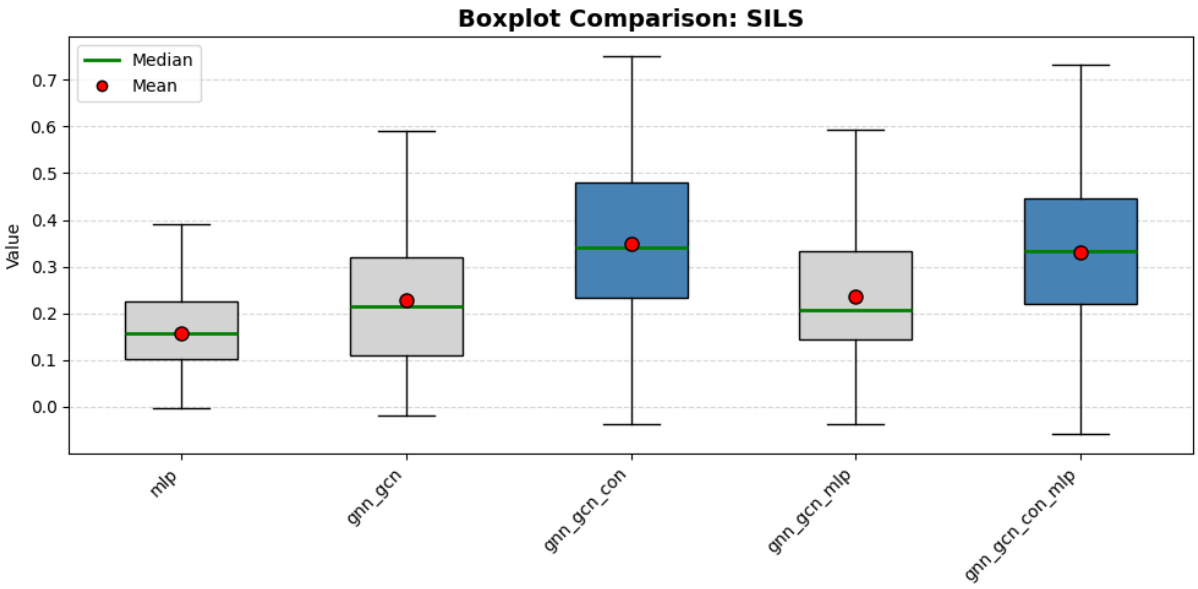
Model A	Model B	P-Value
No significant differences found.		

Since there are no p-values below 0.05, we cannot claim that one model is statistically better than the others. This outcome is likely caused by the combination of a small dataset and very high dimensionality. The fact that the simple MLP performs just as well as the Graph Neural Networks suggests that the graph does not provide useful information.

Silhouette

This metric was calculated on the embeddings right before the classification block. It measures how well the model separates different classes in the latent space.

For MLP it was extracted from the hidden layer, for the GNNs after the encoder; since the loss was not explicitly imposed on the base models, the MLP focuses on class separation, not class clustering.



- **Median:** there is a sharp increase. The contrastive models show a higher median compared to the baselines. This indicates that, in most cases, the contrastive loss forces the model to separate the classes much better.
- **Mean:** the Mean aligns almost perfectly with the Median across all models. This confirms that the improvement is consistent.

- **IQR:** The lower bound of the contrastive models is roughly equal to the median (or even the upper bound) of the baselines. This means that a "below average" result from a contrastive model is still better than a "typical" result from a standard model.
- **Whiskers:** all models have whiskers extending below zero, meaning the clusters overlap and are confused. This is probably caused by some difficult folds.
- **Variance:** the most notable feature is the **height** of the whiskers. The contrastive models have huge variance. This suggests that while the contrastive method creates better separation on average, the quality of the embeddings varies significantly from fold to fold. Notably, the best score is reached by contrastive models only.

The plot proves that the contrastive term successfully "pushes" classes apart. However, it introduces high instability.

MODEL	MEDIAN	MEAN	STD	CI 95%
Raw data	/	0.0283	/	/
mlp	0.1572	0.1573	0.0885	0.0896
gnn_gcn	0.2143	0.2273	0.1503	0.1520
gnn_gcn_con	0.3417	0.3496	0.1736	0.1757
gnn_gcn_mlp	0.2066	0.2352	0.1452	0.1469
gnn_gcn_con_mlp	0.3345	0.3298	0.1697	0.1717

The gnn_gcn_con model has the highest Mean Score. It more than doubles the performance of the standard MLP. Unlike in the accuracy results, here the basic GCN performs clearly better than the MLP, proving that the graph structure is useful for grouping similar nodes.

The Standard Deviation shows a different trend compared to accuracy. The contrastive models have a much higher variance compared to the simple MLP: rotating the embedding is much harder than pushing them away. This means that while the new method improves the separation quality, it makes the model less stable across different folds.

The Confidence Intervals are very wide and overlap, largely due to the high variance. For example, the gnn_gcn_con interval partially overlaps with the MLP range.

As for the p-values:

Model A	Model B	P-Value
mlp	gnn_gcn_con	0.008408
mlp	gnn_gcn_con_mlp	0.016143
gnn_gcn	gnn_gcn_con	0.033156

The P-values confirm that these improvements are real, unlike the accuracy test. This allows me to reject the null hypothesis and confirms that the contrastive models are statistically superior in creating well-separated clusters.

The last key point is that contrastive models organize the data better than the other, yet the final accuracy doesn't increase as much. The problem isn't how the model "sees" the data, but how confident it is when seeing *new* data. It successfully separates the training data, creating a well-ordered space. However, because we have **few data points**, the boundary it draws between classes remains "fragile." The data is well-sorted in space, but the classifier still struggles to draw a stable line that works perfectly on unseen test data.

Overall, given the fact that the loss of contrastive models acts on both metrics, the models are probably clustering correctly all elements except the difficult ones. In this ambiguous zone the graph is not helping so the model are basing the classification only on the data features, hence why the similar performances.

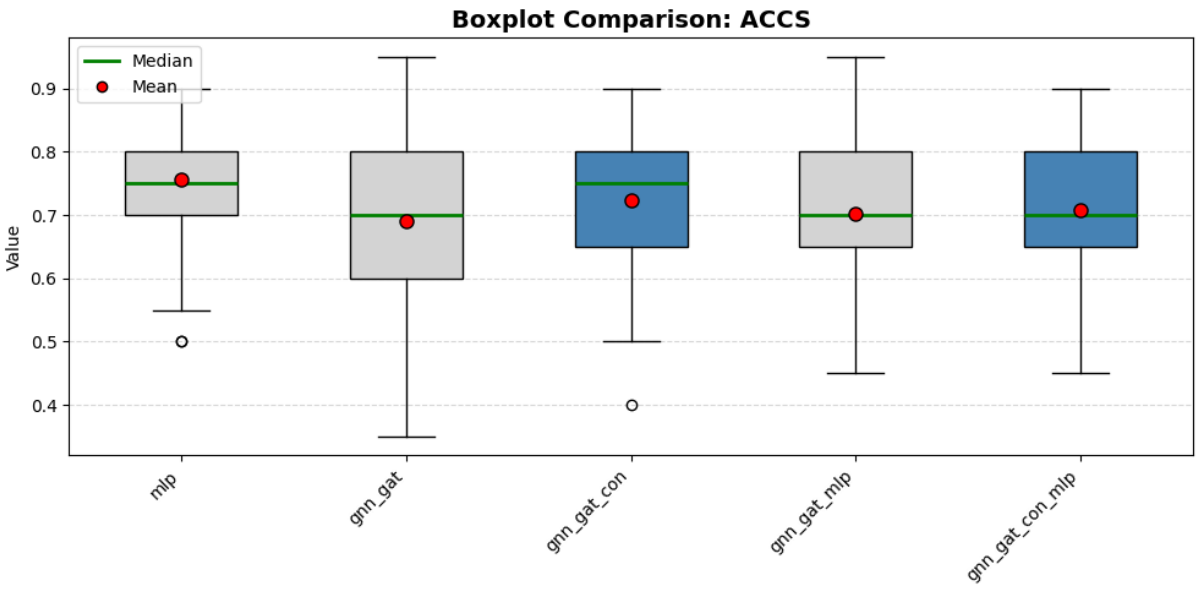
GAT Test

I included GAT to test a specific hypothesis derived from the Homophily analysis. Theoretically, GAT's attention mechanism could filter out this noise by assigning lower weights to dissimilar neighbors. However, I am not excluding the possibility that the excessive amount of parameters will lead to lower performances. This is why I used only a single head for GAT layers.

The setup for this experiment is the same as before, only the type of layer changes.

Since the folds were already generated, there is no need to train again the MLP. I simply executed (almost) the same code.

Accuracy



- **Mean:** MLP achieves the highest average performance, while base GAT models barely reach the 0.70 threshold. Increased parameter count in GAT architectures currently degrades performance, and the contrastive term is not strong enough to offset completely the added complexity.
- **Median:** MLP and gnn_gat_con model share a peak median of 0.75, showing that complexity only matches the baseline at best. Base GAT models perform worse, confirming that the increase in parameters is counterproductive.
- **IQR:** The first quartile (bottom of the box) for MLP aligns with the GAT medians, meaning MLP “bad” results only equal GATs average behavior.
- **Whiskers:** MLP provides the highest performance floor with the shortest lower whisker. GAT models exhibit high volatility and extreme outliers, demonstrating that increased complexity introduces instability without providing a performance gain. MLP is also the only one not performing worse than random (0.5).

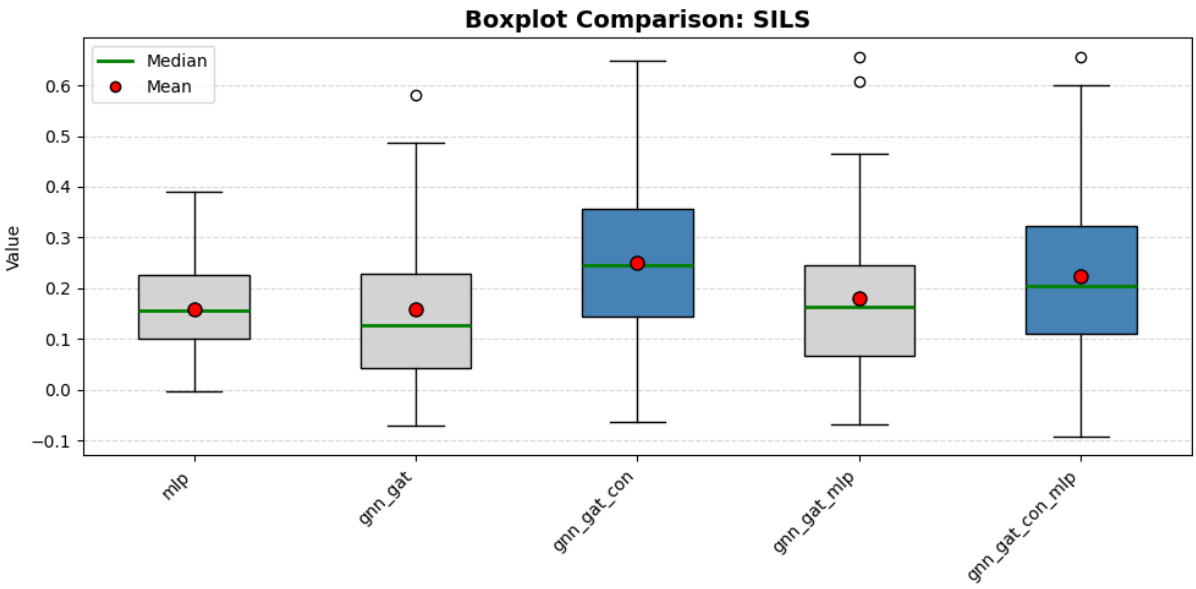
MODEL	MEDIAN	MEAN	STD	CI 95%
mlp	0.7500	0.7555	0.0867	0.0877
gnn_gat	0.7000	0.6910	0.1129	0.1142
gnn_gat_con	0.7500	0.7225	0.1018	0.1030
gnn_gat_mlp	0.7000	0.7025	0.1031	0.1043
gnn_gat_con_mlp	0.7000	0.7080	0.0992	0.1003

Model A	Model B	P-Value
No significant differences found.		

The numbers shows no significant differences between any of the models. Overlapping 95% confidence intervals occur because standard deviations are high across all architectures.

The mlp baseline achieves the highest mean. Complexity in GAT network increase variance without providing a performance boost (even with the added contrastive term).

Silhouette



- **Median:** the contrastive models show a clear increase compared to the baselines. However, this improvement is much smaller than what we saw with the GCN models. The non contrastive performs like the MLP if not (overall) worse.
- **IQR:** the height of boxes for the contrastive models are comparable to the previous experiment, suggesting a similar level of stability.
- **Whiskers:** Here the whiskers of the contrastive models extend more into **negative values**. This means that in many folds, the model fails to separate the classes and assigns them to the wrong class.

MODEL	MEDIAN	MEAN	STD	CI 95%
Raw data	/	0.0283	/	/
mlp	0.1572	0.1573	0.0885	0.0896
gnn_gat	0.1279	0.1577	0.1371	0.1387
gnn_gat_con	0.2446	0.2512	0.1620	0.1639
gnn_gat_mlp	0.1644	0.1813	0.1489	0.1507
gnn_gat_con_mlp	0.2042	0.2239	0.1587	0.1606

The standard gnn_gat performs almost identically to the mlp. This proves that the Attention mechanism is not adding any useful information.

The std for the gnn_gat_con_mlp is very high, it is more than two times instable than the MLP. Other GAT models are not doing much better.

Model A	Model B	P-Value
No significant differences found.		

Unlike the GCN experiment, here no significant differences were found. I cannot claim that the GAT models are statistically different than the MLP.

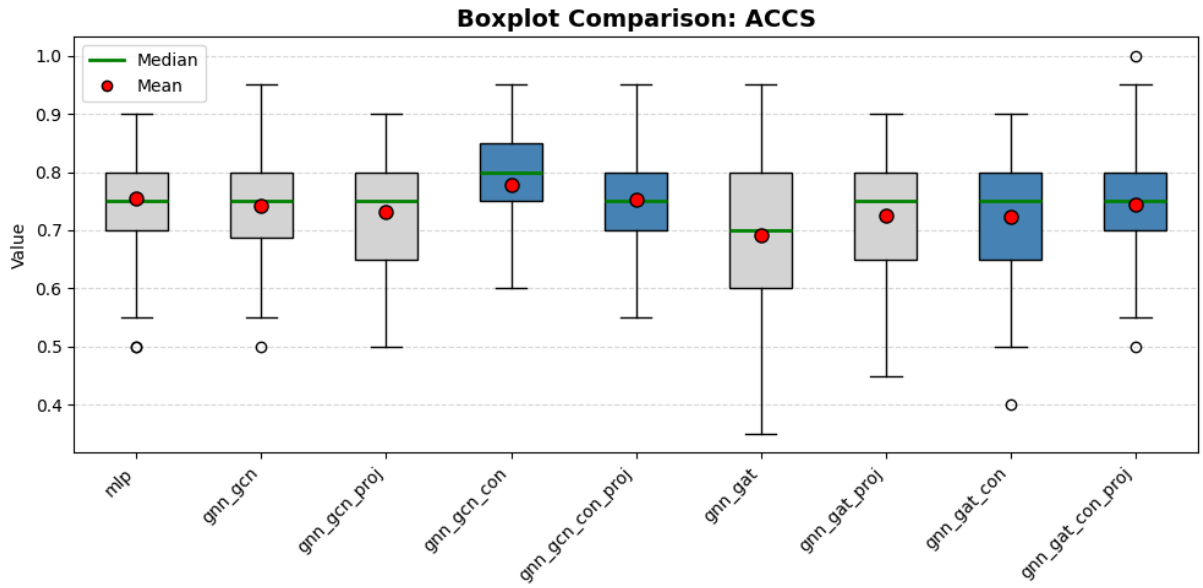
The results demonstrate that using Graph Attention Networks is not useful for this specific task. While contrastive loss attempts to organize the latent space, the attention mechanism fails to learn stable weights. This cancels out the geometric benefits seen earlier. The lack of significant p-values for both Accuracy and Silhouette scores confirms that the MLP architecture remains the most robust choice, offering the best balance between performance and stability.

Noise Reduction – Linear Projection

This experiment was tried to compress the information and reduce the noise the data brings. I implemented a GNN wrapper that adds a linear layer. The number of dimensions selected is half of the embedding's dimensions: this is done to force the model to keep only the most useful information, acting as a simple autoencoder.

The results in this section are provided by selecting only a few configurations (more specifically, by keeping only the linear-layer classifiers, since MLP after the GNN encoder had no positive effect) and comparing the projected version with the non-projected version side by side.

Accuracy



MODEL	MEDIAN	MEAN	STD	CI 95%
mlp	0.7500	0.7555	0.0867	0.0877
gnn_gcn	0.7500	0.7420	0.0968	0.0980
gnn_gcn_proj	0.7500	0.7320	0.0986	0.0998
gnn_gcn_con	0.8000	0.7790	0.0856	0.0866
gnn_gcn_con_proj	0.7500	0.7525	0.0854	0.0864
gnn_gat	0.7000	0.6910	0.1129	0.1142
gnn_gat_proj	0.7500	0.7250	0.0989	0.1000
gnn_gat_con	0.7500	0.7225	0.1018	0.1030
gnn_gat_con_proj	0.7500	0.7445	0.0879	0.0889

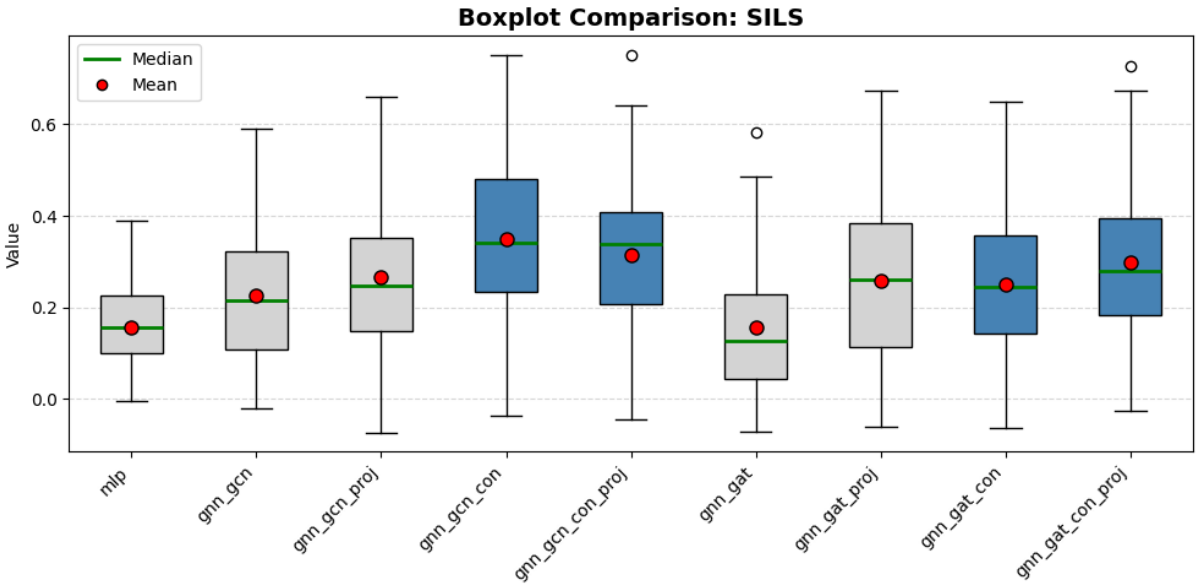
The results present a clear pattern:

- For GCN models: Compressing information is harmful. It causes the model to lose relevant features. This is evident in the boxplots and the table: the projected versions show lower means and medians. They also exhibit wider IQRs, indicating that the models have become more undecided and unstable.

- For GAT models: the projection acts in the opposite way and becomes a helpful tool. Because the GAT attention mechanism can get confused by too much complexity, reducing the dimensions works like a "cleanup" that removes background noise.

Model A	Model B	P-Value
No significant differences found.		

Silhouette



MODEL	MEDIAN	MEAN	STD	CI 95%
Raw data	N/A	0.0283	N/A	N/A
mlp	0.1572	0.1573	0.0885	0.0896
gnn_gcn	0.2143	0.2273	0.1503	0.1520
gnn_gcn_proj	0.2481	0.2674	0.1686	0.1706
gnn_gcn_con	0.3417	0.3496	0.1736	0.1757
gnn_gcn_con_proj	0.3388	0.3146	0.1675	0.1694
gnn_gat	0.1279	0.1577	0.1371	0.1387
gnn_gat_proj	0.2614	0.2590	0.1729	0.1749
gnn_gat_con	0.2446	0.2512	0.1620	0.1639
gnn_gat_con_proj	0.2805	0.2991	0.1562	0.1581

- For GCN models the projection to the contrastive version hurts the results. This happens because the contrastive encoder already organizes the data very efficiently; forcing it through a smaller projection layer creates a "bottleneck" that disrupts the learned structure.
- For GAT models, the projection acts as a filter that removes redundant or "noisy" information, which helps the attention mechanism. The impact is clear in the data: the baseline GAT score nearly doubles when the projection is added.

While the general trends are clear, the p-values for the direct comparison between base models and their projected versions were not statistically significant, so I omitted the comparison table for these specific models.

Conclusions

To conclude, here is a recap of the work performed:

1. I analyzed the dataset to identify potential signals and structural patterns.
2. I designed the graph with the specific objective of maximizing information flow while minimizing the risk of oversmoothing.
3. I tested various architectures and parameter combinations, ensuring a fair comparison by giving all models the same starting conditions.

My findings are:

- The simple MLP proved to be just as effective as the more complex Graph Neural Networks. This suggests that for this specific dataset, the graph structure does not add enough predictive signal to justify the increased complexity.
- The mixed loss successfully regularizes the latent space, creating well-separated clusters (high Silhouette Score). However, this geometric improvement does not translate into better accuracy. The model organizes the data well but still struggles to draw a stable decision boundary for the test data.
- The graph structure provides some local information, but it fails to help with the "hard-to-classify" data points. As my analysis showed, these points often lack local homophily (neighbors have different classes), meaning the graph introduces noise rather than clarity.
- Adding a layer to compress information gave mixed results. It helped GAT models by filtering out noise, but it offered no real benefit for GCN models and sometimes even removed important features. This shows that the models are not struggling with too much repeated information; instead, they simply lack enough data to learn complex patterns.